

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)**

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и облачные технологии**

Направление подготовки **09.03.03 Прикладная информатика**

О Т Ч Е Т

по ЛАБОРАТОРНОЙ РАБОТЕ №3

**Тема: Создание БД PostgreSQL в pgAdmin. Резервное копирование и
восстановление БД (Вариант 4 Учёт выполнения заданий)**

Выполнили: Сатрио Субагио K3240

Преподаватель : Говорова Марина Михайловна

Дата 09.04.2026

Санкт-Петербург 2026

Введение

В рамках этой лабораторной работы мне было необходимо провести анализ предметной области, используя Вариант 4, и построить модель в pdAdmin, используя логическую модель (IDEF1X) из второй лабораторной работы.

Целью этой лабораторной работы было понять информационную структуру, используемую в системе отслеживания задач, определить основные сущности и их взаимосвязи, а также описать ключевые атрибуты и ограничения.

В то время как предыдущий этап был посвящен абстракции данных, в этой лабораторной работе логическая модель (IDEF1X) была преобразована в физическую схему в СУБД PostgreSQL. Эта реализация была направлена на проверку проекта путем реалистичного применения ограничений данных, обеспечения ссылочной целостности между таблицами и тестирования функциональности базы данных посредством оперативного ввода данных и механизмов резервного копирования и восстановления.

Краткое описание предметной области (вариант 4)

Система предназначена для учёта выполнения заданий внутри проектов, которые создаются на основе договоров с организациями.

Внутри компании работают сотрудники, которые состоят в отделах и занимают определённые должности. Для каждого сотрудника фиксируется кадровая история.

Каждый проект связан с договором, а внутри проекта создаются задания.

Задания могут выполняться разными сотрудниками — для этого вводится сущность «Участие», которая отражает назначение исполнителя на конкретное задание.

Также ведётся контроль выполнения заданий.

Основные сущности предметной области:

- Отдел
- Должность
- Сотрудник
- Кадровая история
- Сотрудник на проект
- Организация
- Договор
- Проект
- Задание
- Контроль
- Участие (ассоциативная)

Даталогическая модель (IDEF1X)

Ниже представлена даталогическая модель в нотации IDEF1X, отображающая связи “родитель–потомок”, обязательность участия и структуру будущей БД.

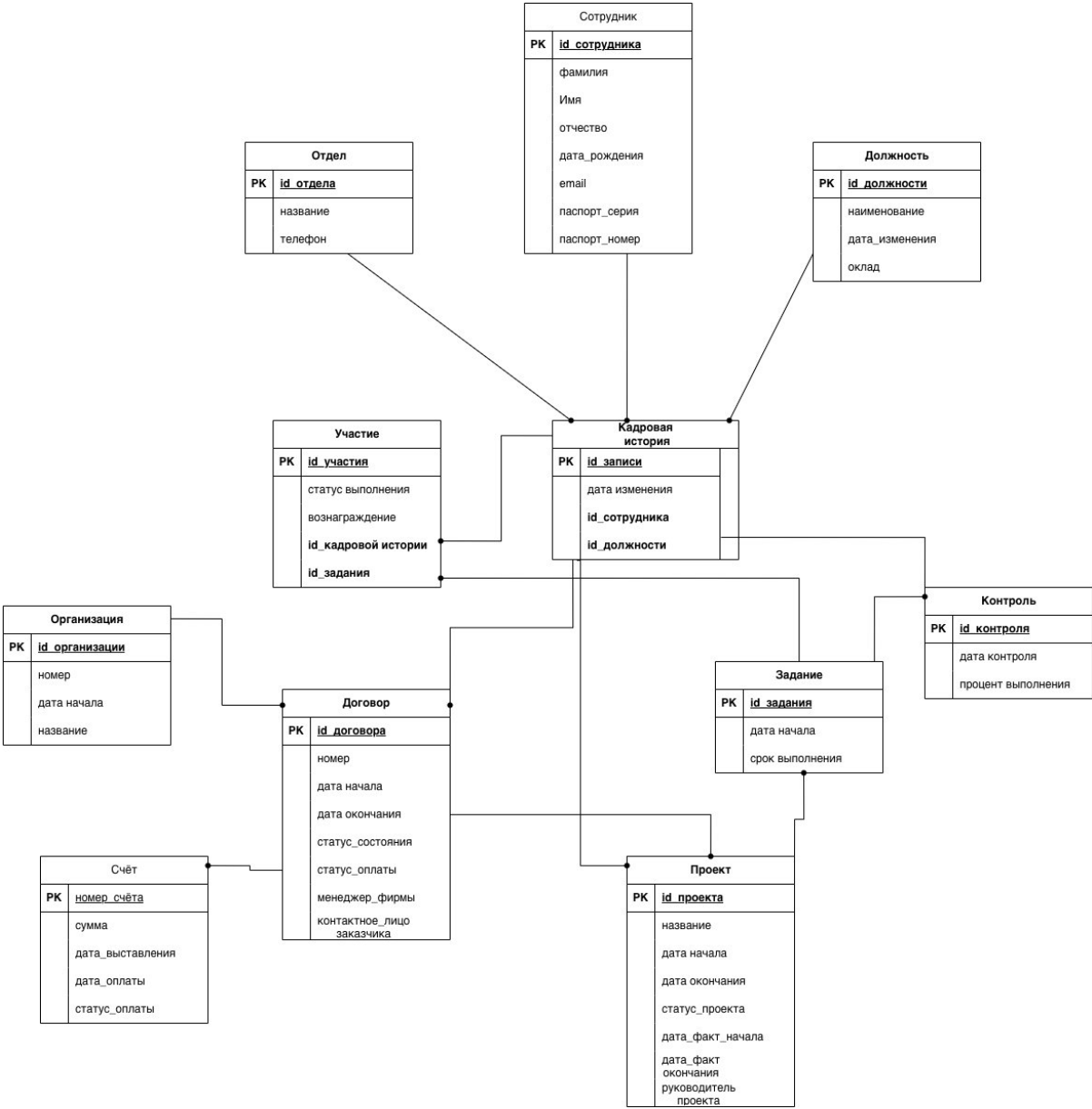


Рисунок 1 — Даталогическая модель IDEF1X

Таблица описания данных

Таблица 1 — Описание атрибутов сущностей

Сущность: Отдел

Атрибут	Тип данных	PK	FK	Обяз.	ограничение
id_отдела	INTEGER	+		+	уникален
название	VARCHAR(100)			+	допустимы: кириллица, латиница, пробел, дефис, цифры
телефон	VARCHAR(20)				маска ввода ?

Сущность: Должность

Атрибут	Тип данных	PK	FK	Обяз.	ограничение
id_должности	INTEGER	+		Да	

наименование	VARCHAR(100)			Да	
оклад				да	> 0

Сущность: Сотрудник

Атрибут	Тип данных	PK	FK	Обяз.	ограничение
id_сотрудника	INTEGER	+		Да	
фамилия	VARCHAR(100)			Да	
имя	VARCHAR(100)			Да	
отчество	VARCHAR(100)			Нет	NULL допускается
дата_рождения	DATE	Нет		Нет	> 01.01.1930
паспорт_серия	VARCHAR(100)			Да	допустимые латиница цифры дефис

паспорт_номер	VARCHAR(100)			Да	
---------------	--------------	--	--	----	--

Сущность: Кадровая история

Атрибут	Тип данных	PK	FK	Обяз.	ограничение
id_истории	INTEGER	+		Да	
id_сотрудника	INTEGER		+	Да	FK → Сотрудник
id_должности	INTEGER		+	Да	FK → Должность
дата_начала	DATE			Да	< дата окончания
дата_окончания	DATE			Нет	

Сущность: Организация

Атрибут	Тип данных	PK	FK	Обяз.	ограничение
id_организации	INTEGER	+		Да	

название	VARCHAR(150)			Да	допустимы: кириллица, латиница, пробел, дефис, цифры
адрес	VARCHAR(200)			Нет	
контактное лицо	VARCHAR(150)			Нет	

Сущность: Договор

Атрибут	Тип данных	PK	FK	Обяз.	ограничение
id_договора	INTEGER	+		Да	
номер	VARCHAR(50)			Да	
дата_начала	DATE			да	< дата окончания
дата_окончания	DATE			Нет	

id_организации	INTEGER		+	Да	FK → договор
----------------	---------	--	---	----	--------------

Сущность: Проект

Атрибут	Тип данных	PK	FK	Обяз.	ограничение
id_проекта	INTEGER	+		Да	
название	VARCHAR(150)			Да	допустимы: кириллица, латиница, пробел, дефис, цифры
дата_начала	DATE			Да	
дата_окончания	DATE			Нет	
id_договора	INTEGER		+	Да	FK → проект

Сущность: Задание

Атрибут	Тип данных	PK	FK	Обяз.	ограничение
id_задания	INTEGER	+		Да	

дата_начала	DATE			Да	< дата окончания
срок выполнения	DATE			Да	
id_проекта	INTEGER		+	Да	FK → задание

Сущность: Контроль

Атрибут	Тип данных	PK	FK	Обяз.	ограничение
id_контроля	INTEGER	+		Да	
дата_контроля	DATE			Да	
процент выполнения	INTEGER			Да	
id_задания	INTEGER		+	Да	FK → участие

Сущность: Участие (ассоциативная)

Атрибут	Тип данных	PK	FK	Обяз.	ограничение
id_участия	INTEGER	+		Да	FK → Сотрудник_проект
id_сотрудник-проекта	INTEGER		+	Да	FK → участие

id_задания	INTEGER		+	Да	FK → участие
------------	---------	--	---	----	-----------------

Реализация физической модели в СУБД

Переход от логического проектирования к физической реализации в PostgreSQL 1X с использованием pgAdmin 4 включил в себя следующие ключевые этапы:

1. Трансформация атрибутов в типы данных

На этапе реализации для каждого атрибута из модели IDEF1X были подобраны соответствующие типы данных PostgreSQL:

Для идентификаторов и суррогатных ключей использован тип SERIAL или INT с автоинкрементом.

Для текстовых описаний — VARCHAR(N) или TEXT.

Для дат и временных меток — DATE или TIMESTAMP.

2. Обеспечение целостности данных

Для точного соответствия правилам предметной области (Вариант 4) на уровне таблиц были установлены ограничения:

Primary Key: для обеспечения уникальности записей каждой сущности.

Foreign Key: для реализации связей, спроектированных в IDEF1X, и обеспечения ссылочной целостности.

Check Constraints: для логической проверки данных (например, чтобы дата завершения задания не была раньше даты постановки).

Not Null / Unique: для обязательных полей и уникальных бизнес-ключей.

3. Результаты

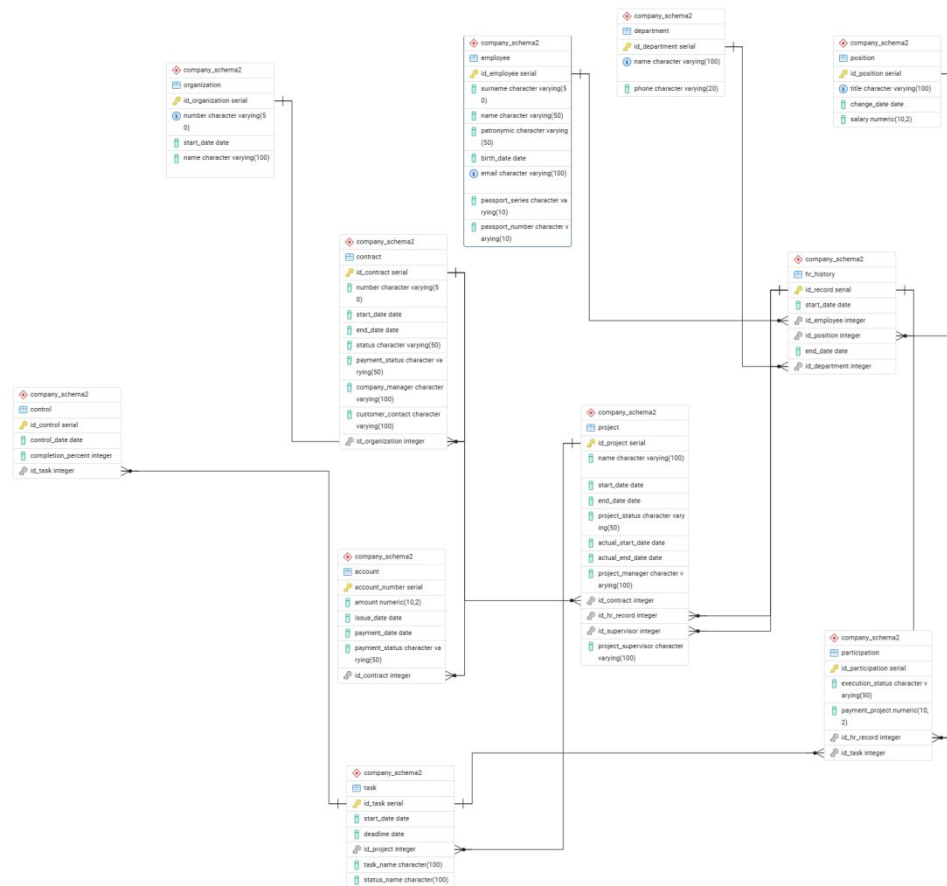


Рисунок 2 — Логическая модель (ERD из pgAdmin)

```

-- =====
-- ЛАБОРАТОРНАЯ РАБОТА №3.2
-- Скрипт создания и заполнения базы данных (Вариант 4)
-- Студент: Сатрио Брахманторо Ади
-- =====

BEGIN;

-- 1. СОЗДАНИЕ СХЕМЫ (Penyusunan Skema)
CREATE SCHEMA IF NOT EXISTS company_schema2;

-- 2. СОЗДАНИЕ ТАБЛИЦ И ОГРАНИЧЕНИЙ (Pembuatan Tabel & Constraints)

-- Таблица Должностей (Tabel Jabatan)
CREATE TABLE IF NOT EXISTS company_schema2."position"
  
```

```
(
    id_position serial NOT NULL,
    title character varying(100) NOT NULL,
    change_date date,
    salary numeric(10, 2) CHECK (salary > 0), -- Constraint: Check (Gaji harus positif)
    CONSTRAINT pk_position PRIMARY KEY (id_position),
    CONSTRAINT uk_position_title UNIQUE (title) -- Constraint: Unique
);
```

-- Таблица Отделов (Tabel Departemen)

```
CREATE TABLE IF NOT EXISTS company_schema2.department
```

```
(
    id_department serial NOT NULL,
    name character varying(100) NOT NULL,
    phone character varying(20),
    CONSTRAINT pk_department PRIMARY KEY (id_department),
    CONSTRAINT uk_department_name UNIQUE (name)
);
```

-- Таблица Сотрудников (Tabel Karyawan)

```
CREATE TABLE IF NOT EXISTS company_schema2.employee
```

```
(
    id_employee serial NOT NULL,
    surname character varying(50) NOT NULL,
    name character varying(50) NOT NULL,
    patronymic character varying(50),
    birth_date date,
    email character varying(100),
    passport_series character varying(10),
    passport_number character varying(10),
    CONSTRAINT pk_employee PRIMARY KEY (id_employee),
    CONSTRAINT uk_employee_email UNIQUE (email)
);
```

-- Таблица Истории Кадров (Tabel Riwayat HR)

```
CREATE TABLE IF NOT EXISTS company_schema2.hr_history
```

```
(
    id_record serial NOT NULL,
    start_date date NOT NULL,
    end_date date,
```

```

id_employee integer NOT NULL,
id_position integer NOT NULL,
id_department integer NOT NULL,
CONSTRAINT pk_hr_history PRIMARY KEY (id_record)
);

-- Таблица Организаций (Tabel Organisasi)
CREATE TABLE IF NOT EXISTS company_schema2.organization
(
    id_organization serial NOT NULL,
    "number" character varying(50),
    start_date date,
    name character varying(100) NOT NULL,
    CONSTRAINT pk_organization PRIMARY KEY (id_organization),
    CONSTRAINT uk_organization_number UNIQUE ("number")
);

-- Таблица Контрактов (Tabel Kontrak)
CREATE TABLE IF NOT EXISTS company_schema2.contract
(
    id_contract serial NOT NULL,
    "number" character varying(50),
    start_date date,
    end_date date,
    status character varying(50),
    payment_status character varying(50),
    company_manager character varying(100),
    customer_contact character varying(100),
    id_organization integer,
    CONSTRAINT pk_contract PRIMARY KEY (id_contract)
);

-- Таблица Счетов (Tabel Tagihan/Account)
CREATE TABLE IF NOT EXISTS company_schema2.account
(
    account_number serial NOT NULL,
    amount numeric(10, 2),
    issue_date date,
    payment_date date,
    payment_status character varying(50),

```

```
id_contract integer,  
CONSTRAINT pk_account PRIMARY KEY (account_number)  
);
```

-- Таблица Проектов (Tabel Proyek)

```
CREATE TABLE IF NOT EXISTS company_schema2.project  
(  
    id_project serial NOT NULL,  
    name character varying(100) NOT NULL,  
    start_date date,  
    end_date date,  
    project_status character varying(50),  
    id_contract integer,  
    id_hr_record integer, -- Менеджер проекта  
    CONSTRAINT pk_project PRIMARY KEY (id_project)  
);
```

-- Таблица Задач (Tabel Tugas/Task)

```
CREATE TABLE IF NOT EXISTS company_schema2.task  
(  
    id_task serial NOT NULL,  
    task_name character varying(100),  
    start_date date,  
    deadline date,  
    status_name character varying(100),  
    id_project integer,  
    CONSTRAINT pk_task PRIMARY KEY (id_task)  
);
```

-- 3. УСТАНОВКА ВНЕШНИХ КЛЮЧЕЙ (Foreign Keys)

```
ALTER TABLE company_schema2.hr_history  
    ADD CONSTRAINT fk_hr_employee FOREIGN KEY (id_employee) REFERENCES  
company_schema2.employee (id_employee) ON UPDATE CASCADE ON DELETE CASCADE,  
    ADD CONSTRAINT fk_hr_position FOREIGN KEY (id_position) REFERENCES  
company_schema2."position" (id_position) ON UPDATE CASCADE,  
    ADD CONSTRAINT fk_hr_department FOREIGN KEY (id_department) REFERENCES  
company_schema2.department (id_department) ON UPDATE CASCADE;
```

```
ALTER TABLE company_schema2.contract
```



```
ADD CONSTRAINT fk_contract_org FOREIGN KEY (id_organization) REFERENCES  
company_schema2.organization (id_organization);
```

```
ALTER TABLE company_schema2.project
```

```
ADD CONSTRAINT fk_project_contract FOREIGN KEY (id_contract) REFERENCES  
company_schema2.contract (id_contract);
```

```
ALTER TABLE company_schema2.task
```

```
ADD CONSTRAINT fk_task_project FOREIGN KEY (id_project) REFERENCES  
company_schema2.project (id_project);
```

-- 4. ЗАПОЛНЕНИЕ ТАБЛИЦ РАБОЧИМИ ДАННЫМИ (Pengisian Data - Wajib ada di laporan)

-- Data Jabatan

```
INSERT INTO company_schema2."position" (title, salary) VALUES  
( 'Senior Developer', 150000),  
( 'Project Manager', 180000),  
( 'Analyst', 120000);
```

-- Data Departemen

```
INSERT INTO company_schema2.department (name, phone) VALUES  
( 'IT Development', '555-0101'),  
( 'Analytics Dept', '555-0102');
```

-- Data Karyawan

```
INSERT INTO company_schema2.employee (surname, name, email) VALUES  
( 'Ivanov', 'Ivan', 'ivanov@company.ru'),  
( 'Petrova', 'Elena', 'petrova@company.ru');
```

-- Data Riwayat HR (Menghubungkan Karyawan ke Jabatan & Dept)

```
INSERT INTO company_schema2.hr_history (start_date, id_employee, id_position, id_department)  
VALUES  
( '2025-01-01', 1, 1, 1),  
( '2025-01-15', 2, 2, 2);
```

-- Data Organisasi & Kontrak

```
INSERT INTO company_schema2.organization (name, "number") VALUES ('Gazprom', 'ORG-001');  
INSERT INTO company_schema2.contract ("number", start_date, id_organization) VALUES  
( 'CONT-2026-01', '2026-02-01', 1);
```

-- Data Proyek & Tugas

```
INSERT INTO company_schema2.project (name, start_date, id_contract) VALUES ('AI Monitoring',  
'2026-03-01', 1);
```

```
INSERT INTO company_schema2.task (task_name, id_project, status_name) VALUES ('Database  
Design', 1, 'In Progress');
```

```
COMMIT;
```

4. Наполнение и верификация

Заполнение таблиц «рабочими данными» позволило проверить корректность работы установленных связей и ограничений в реальных условиях эксплуатации.

4. Резервное копирование и сохранность

Заключительным этапом стало освоение инструментов администрирования: создание дампов (в форматах Custom и Plain Text). Это критически важный навык для обеспечения отказоустойчивости системы и возможности миграции базы данных между серверами.

Заключение

В ходе выполнения лабораторной работы была успешно реализована физическая модель базы данных в СУБД PostgreSQL 1X на основе спроектированной ранее логической модели IDEF1X. С помощью графического интерфейса pgAdmin 4 была создана рабочая схема базы данных, в которой реализованы все необходимые таблицы с учетом трансформации составных ключей из инфологической модели в простые суррогатные ключи для повышения эффективности обработки запросов и индексации.

Особое внимание в работе было уделено обеспечению целостности данных путем настройки программных ограничений Primary Key, Foreign Key, Unique и Check. Это позволило формализовать бизнес-правила предметной области непосредственно на уровне базы данных, исключив возможность появления логических противоречий. Процесс ручного заполнения таблиц рабочими данными с использованием SQL-запросов подтвердил корректность установленных связей и работоспособность всей структуры в реальных условиях.

Заключительным этапом работы стало освоение инструментов администрирования, включая процедуры резервного копирования в форматах Custom и Plain Text. Успешное восстановление базы данных из дампа позволило на практике закрепить навыки обеспечения сохранности информации и мобильности данных. Таким образом, выполнение работы сформировало комплексное понимание цикла разработки базы данных — от концептуального проектирования до физической реализации и поддержки.